



รวมชุดคำสั่ง AVR Assembly ครบทุกคำสั่ง และคำอธิบายโค้ดตัวอย่างบรรทัดต่อบรรทัด (Line-by-Line Code Explanations)



PART 1: สรุปเน้นสอบตอนเช้า (ปิดตำรา - CLOSED BOOK)

1.1 K-Map 4x4, Full Adder & CPU Pipeline

- DeMorgan: $(A \cdot B)' = A' + B'$, Minterm (m_i) ใน SOP เอาต์พุต=1
- Full Adder & Overflow: $Sum = A \oplus B \oplus Cin, V = Cin(MSB) \oplus Cout(MSB)$
- Flip-Flops: D ($Q(t+1)=D$), JK ($Q(t+1)=J \cdot Q' + K' \cdot Q$), T ($Q(t+1)=T \oplus Q$)
- Control Unit: Hardwired FSM (เร็ว ยืดหยุ่นน้อย เหมาะ RISC) vs Microprogrammed ROM Store (ช้ากว่า แก้ไขง่าย เหมาะ CISC)



2.1 คลังชุดคำสั่งภาษา AVR Assembly ครบทุกคำสั่ง (All 40+ Instructions Table)

หมวดคำสั่ง	คำสั่ง (Opcode)	รูปแบบ (Syntax)	การทำงาน (Operation) & คำอธิบาย
ย้ายข้อมูล	LDI / MOV / MOVW	LDI Rd, K	โหลดค่าคงที่ K ลง Rd (R16 ถึง R31) / 'MOVW' ก๊อปปี้ 16 บิต
	LDS / STS	LDS Rd, k / STS k, Rr	อ่าน/เขียนข้อมูลตรงกับแอดเดรส k ใน Data SRAM
	IN / OUT	IN Rd, A / OUT A, Rr	อ่าน/เขียนข้อมูลพอร์ต I/O Register A (เช่น PORTB, PINC)
	PUSH / POP / LPM	PUSH Rr / LPM Rd, Z	เก็บ/ดึง Stack / อ่านข้อมูล 1 ไบต์จาก Flash ROM ชี้โดย Z Pointer
คำนวณ ALU	ADD / ADC	ADD Rd, Rr	บวก Rd = Rd + Rr ('ADC' บวก Carry C เพิ่มอีก 1)
	SUB / SUBI / SBC	SUBI Rd, K	ลบด้วยค่าคงที่ K หรือลบรวมตัวยืม Carry C
	INC / DEC / NEG	INC Rd / NEG Rd	เพิ่ม/ลดค่าทีละ 1 / หาค่า 2's Complement เปลี่ยนเครื่องหมาย
	MUL / MULS / CPI	MUL Rd, Rr	คูณ 8 บิต ผลลัพธ์ 16 บิตในคู่ 'R1:R0' / 'CPI' เปรียบเทียบเซต SREG
ลอจิก & บิต	ANDI / ORI / EOR	ANDI Rd, K	ลอจิก AND Mask บิต / ORI เซตบิต 1 / EOR XOR บิต
	SBI / CBI	SBI A, b / CBI A, b	Set Bit 1 หรือ Clear Bit 0 ของพอร์ต I/O Register A ขา b
	SBIS / SBIC	SBIS A, b	ข้าม 1 คำสั่งถ้าบิต I/O b เป็น 1 ('SBIS') หรือ 0 ('SBIC')
กระโดดเงื่อนไข	RJMP / CALL / RET	CALL Subroutine	กระโดดไร้เงื่อนไข / เรียกฟังก์ชันย่อย และดึงกลับด้วย 'RET'
	BREQ / BRNE	BRNE label	กระโดดเมื่อ Z=1 (เท่ากัน) หรือ Z=0 (ไม่เท่ากัน)
	BRLO / BRSH	BRLO label	กระโดดเมื่อ C=1 (น้อยกว่า) หรือ C=0 (มากกว่า/เท่ากับ)
	SEI / CLI / RETI	SEI / RETI	เปิด Global Interrupt (I=1) / ดึง PC คืนจาก Interrupt ISR

2.1.1 ตารางถอดรหัสบิต Opcode และ Operand 16 บิตของ AVR (16-Bit Instruction Bit-Encoding) OPCODE BIT MAP

จำแนกบิต Opcode (รหัสคำสั่ง), บิตระบุรีจิสเตอร์เป้าหมาย (SdS), รีจิสเตอร์ต้นทาง (SrS), แอดเดรส I/O (SAS), ค่าคงที่ (SKS), ตำแหน่งบิต (SbS), และ Relative Offset (SkS) ในคำสั่งขนาด 16 บิต (Bit 15 ถึง Bit 0):

หมวดคำสั่ง	คำสั่ง (Instruction)	โครงสร้างรหัสบิต 16 บิต (16-Bit Opcode Encoding)	การกระจายบิต Opcode (Bit 15 - Bit 0)	รายละเอียดความหมายของบิตแต่ละส่วน
คณิตศาสตร์ 2 รีจิสเตอร์	ADD Rd, Rr	0000 11rd dddd rrrr	[15:10] Opcode 000011 [9,3:0] Rr r rrrrr [8:4] Rd d dddd	• Bit 15-10: Opcode คำสั่ง ADD (000011) • Bit 8-4: Rd 5 บิต (\$R0-R_{31}\$) • Bit 9,3-0: Rr 5 บิต (\$R0-R_{31}\$)
	SUB Rd, Rr	0001 10rd dddd rrrr	[15:10] Opcode 000110 [9,3:0] Rr r rrrrr [8:4] Rd d dddd	• Bit 15-10: Opcode คำสั่ง SUB (000110) • Bit 8-4: Rd 5 บิต • Bit 9,3-0: Rr 5 บิต
โหลดค่าคงที่	LDI Rd, K	1110 KKKK dddd KKKK	[15:12] Opcode 1110 [11:8] \$K_{7-4}\$ KKKK [7:4] Rd dddd [3:0] \$K_{3-0}\$ KKKK	• Bit 15-12: Opcode คำสั่ง LDI (1110) • Bit 11-8: ค่าคงที่ \$K\$ 4 บิตสูง (\$K_{7-4}\$) • Bit 7-4: Rd 4 บิต (R16 ถึง R31) • Bit 3-0: ค่าคงที่ \$K\$ 4 บิตต่ำ (\$K_{3-0}\$)
อ่าน/เขียน I/O	IN Rd, A	1011 0AA dddd AAAA	[15:11] Opcode 10110 [10:9,3:0] Address \$A\$ AAAAAA [8:4] Rd dddddd	• Bit 15-11: Opcode คำสั่ง IN (10110) • Bit 10-9,3-0: I/O Address \$A\$ 6 บิต (0x00-0x3F) • Bit 8-4: Rd 5 บิต (\$R0-R_{31}\$)
	OUT A, Rr	1011 1AAr rrrr AAAA	[15:11] Opcode 10111 [10:9,3:0] Address \$A\$ AAAAAA [8:4] Rr rrrrrr	• Bit 15-11: Opcode คำสั่ง OUT (10111) • Bit 10-9,3-0: I/O Address \$A\$ 6 บิต (0x00-0x3F) • Bit 8-4: Rr 5 บิต (\$R0-R_{31}\$)
จัดการบิต I/O	SBI A, b	1001 1010 AAAA Abbb	[15:8] Opcode 10011010 [7:3] Address \$A\$ AAAAA [2:0] Bit pos \$b\$ bbb	• Bit 15-8: Opcode คำสั่ง SBI (10011010) • Bit 7-3: I/O Address \$A\$ 5 บิต (0x00-0x1F) • Bit 2-0: ตำแหน่งบิต \$b\$ 3 บิต (0 ถึง 7)
	CBI A, b	1001 1000 AAAA Abbb	[15:8] Opcode 10011000 [7:3] Address \$A\$ AAAAA [2:0] Bit pos \$b\$ bbb	• Bit 15-8: Opcode คำสั่ง CBI (10011000) • Bit 7-3: I/O Address \$A\$ 5 บิต (0x00-0x1F) • Bit 2-0: ตำแหน่งบิต \$b\$ 3 บิต (0 ถึง 7)
กระโดด เงื่อนไข	BREQ k	1111 00kk kkkk k000	[15:10] Opcode 111100 [9:3] Offset \$k\$ kkkkkkk [2:0] Flag \$Z=1\$ 000	• Bit 15-10: Opcode คำสั่ง BRBS (111100) • Bit 9-3: Relative Offset \$k\$ คัดเครื่องหมาย 7 บิต • Bit 2-0: Flag Bit (000 = ธง Z=1)
	BRNE k	1111 01kk kkkk k000	[15:10] Opcode 111101 [9:3] Offset \$k\$ kkkkkkk [2:0] Flag \$Z=0\$ 000	• Bit 15-10: Opcode คำสั่ง BRBC (111101) • Bit 9-3: Relative Offset \$k\$ คัดเครื่องหมาย 7 บิต • Bit 2-0: Flag Bit (000 = ธง Z=0)

หมวดคำสั่ง	คำสั่ง (Instruction)	โครงสร้างรหัส 16 บิต (16-Bit Opcode Encoding)	การกระจายบิต Opcode (Bit 15 - Bit 0)	รายละเอียดความหมายของบิตแต่ละส่วน
กระโดด สัมพันธ์	RJMP k	1100 kkkk kkkk kkkk	[15:12] Opcode 1100 [11:0] Offset \$k\$ kkkkkkkkkkkk	• Bit 15-12: Opcode คำสั่ง RJMP (1100) • Bit 11-0: Relative Offset \$k\$ 12 บิต (-2048 ถึง +2047)
	RCALL k	1101 kkkk kkkk kkkk	[15:12] Opcode 1101 [11:0] Offset \$k\$ kkkkkkkkkkkk	• Bit 15-12: Opcode คำสั่ง RCALL (1101) • Bit 11-0: Relative Offset \$k\$ 12 บิต (-2048 ถึง +2047)

2.2 อธิบายโค้ดตัวอย่างบรรทัดต่อบรรทัด (Line-by-Line Code Explanations)

1. การตั้งค่า Stack Pointer (SPH:SPL = RAMEND)

```
.INCLUDE "M328PDEF.INC"  
.ORG 0x0000  
LDI R16, HIGH(RAMEND)  
OUT SPH, R16  
LDI R16, LOW(RAMEND)  
OUT SPL, R16
```

💡 คำอธิบายบรรทัดต่อบรรทัด:

- **.INCLUDE & .ORG:** ดึงไฟล์นิยามแอดเดรสชิป และตั้งจุดเริ่มต้นที่ Flash 0x0000
- **LDI R16, HIGH(RAMEND) & OUT SPH, R16:** ตั้ง 8 บิตสูงของ RAMEND (0x08) ขึ้นแอดเดรสสูงสุด SRAM ลง SPH
- **LDI R16, LOW(RAMEND) & OUT SPL, R16:** ตั้ง 8 บิตต่ำของ RAMEND (0xFF) ลง SPL ให้ Stack พร้อมใช้งาน

2. อัลกอริทึมแปลง Hex เป็น BCD 2 หลัก (ลบเข้าด้วย 10)

```
LDI R16, 0x45      ; ค่าอินพุต 0x45 = 69 ทศนิยม  
CLR R17            ; R17 เก็บหลักสิบ (Tens)  
  
BCD_LOOP:  
    CPI R16, 10  
    BRLO BCD_DONE  
    SUBI R16, 10    ; ลบออกทีละ 10  
    INC R17         ; เพิ่มหลักสิบ  
    RJMP BCD_LOOP  
  
BCD_DONE:  
    SWAP R17  
    OR R17, R16     ; รวมหลักสิบและหลักหน่วย (0x69)  
    OUT PORTB, R17
```

💡 คำอธิบายบรรทัดต่อบรรทัด:

- **LDI R16, 0x45 & CLR R17:** กำหนดค่าอินพุตเริ่มต้น 69 และล้างค่าหลักสิบ R17 เป็น 0
- **CPI R16, 10 & BRLO BCD_DONE:** เช็คว่าค่าเหลือถึง 10 ไหม ถ้าน้อยกว่า 10 จบ loop (เศษใน R16 คือหลักหน่วย)
- **SUBI R16, 10 & INC R17:** ลบออกทีละ 10 และเพิ่มค่านับหลักสิบ R17 ขึ้นทีละ 1
- **SWAP R17 & OR R17, R16:** ย้ายหลักสิบขึ้นไป 4 บิตสูง (SWAP) แล้ว OR รวมหลักหน่วย ได้ BCD 0x69

3. โค้ดแล็บจริง: AssemblyBlink1.asm (ควบคุมไฟกะพริบพร้อมกัน 2 พอร์ต)

```
;=====
; Title: AssemblyBlink1.asm
; Created: 11/22/2013
; Author: khatathapswatdipisal
; Description: Blink all LEDs on PORTB and PORTD simultaneously
;=====

.nolist
.include "m328pdef.inc"
.list

; Start at main after reset
rjmp main

main:
    ldi r16, 0xFF          ; Load 0xFF into R16 (all bits set to 1)
    out DDRB, r16          ; Configure Port B pins as Outputs
    out DDRD, r16          ; Configure Port D pins as Outputs
    ldi r16, 0x00          ; Clear R16 to 0x00

loop:
    com r16                ; Complement R16 (toggle bits between 0x00 and 0xFF)
    out PORTB, r16         ; Write R16 to Port B
    out PORTD, r16         ; Write R16 to Port D
    call MY_DELAY          ; Delay for simulation timing
    rjmp loop              ; Loop indefinitely

; Delay Subroutine
MY_DELAY:
    ldi r20, 8
L1:
    ldi r21, 200
L2:
    ldi r22, 250
L3:
    dec r22                ; Decrement R22
    brne L3                ; Loop if R22 != 0
    dec r21                ; Decrement R21
    brne L2                ; Loop if R21 != 0
    dec r20                ; Decrement R20
    brne L1                ; Loop if R20 != 0
    ret                    ; Return from subroutine
```

💡 คำอธิบายบรรทัดต่อบรรทัด:

- **out DDRB/DDRD, r16:** ตั้งพิน PORTB และ PORTD ทั้งหมดเป็นเอาต์พุต
- **com r16 & out PORTB/PORTD, r16:** กลับบิต R16 สลับ 0x00 → 0xFF ทำให้ไฟกะพริบติดดับพร้อมกัน
- **call MY_DELAY:** เรียกฟังก์ชันย่อยหน่วงเวลา 3 ลูปซ้อนกัน (R20, R21, R22)